

FACULTAD DE INFORMÁTICA

Máster en Internet de las Cosas
2025-2026



Asignatura:

Diseño de Infraestructura Inteligente para IOT (DII)

Práctica:

Trabajos individuales grupales

Alumno:

Ariel Gámez Díaz

Sumario

Estado del Arte.....	3
Análisis Comparativo de Flujos de Trabajo en Desarrollo de Smart Contracts: Remix IDE vs. Foundry Scripting.....	4
1.0 Flujo de Trabajo 1: El Enfoque Visual e Interactivo con Remix Desktop.....	4
Ventajas del Enfoque Visual.....	4
Desventajas Intrínsecas.....	5
2.0 Flujo de Trabajo 2: El Enfoque de Ingeniería "Code-First" con Foundry.....	5
Ventajas del Enfoque Foundry.....	6
Prerrequisitos del Enfoque.....	7
3.0 Comparativa Directa: Remix vs. Foundry.....	7
Resumen Comparativo de Flujos de Trabajo.....	7
Análisis Detallado de las Características.....	8
4.0 El Enfoque Híbrido: La Estrategia Óptima para el Desarrollador Moderno.....	9
5.0 Anexo Técnico: Por Qué Anvil es Superior a Geth para el Desarrollo Local.....	9
5.1 Modelo de Minado: Determinismo vs. Realismo.....	10
5.2 "God Mode": Cheatcodes y Control Total del Estado.....	10
5.3 Configuración Cero vs. Bloque Génesis.....	10
5.4 Forking de Mainnet: La Capacidad Definitiva para Pruebas de Integración.....	11
Conclusión y Recomendaciones Finales.....	12



Estado del Arte

En la ingeniería de smart contracts, la selección del conjunto de herramientas es una decisión arquitectónica fundamental que define el curso de un proyecto. Si bien el componente subyacente, como un nodo blockchain local ejecutado en Docker, puede ser idéntico en distintas configuraciones, la metodología de interacción —ya sea a través de una interfaz visual o mediante scripting programático— redefine radicalmente la eficiencia, la reproducibilidad y la escalabilidad del proceso. Una elección informada reduce la superficie de ataque del proceso de desarrollo al minimizar el error humano y maximizar la automatización.

Este documento tiene como propósito analizar y contrastar objetivamente dos enfoques predominantes: el entorno visual e interactivo de **Remix Desktop** y el flujo de trabajo de ingeniería "code-first" que ofrece **Foundry**.

Dirigido a desarrolladores de blockchain profesionales, este análisis busca proporcionar una guía clara para seleccionar el conjunto de herramientas más adecuado según la complejidad del proyecto, la fase de desarrollo y los objetivos del equipo. La meta es capacitar al lector para tomar decisiones informadas que optimicen tanto la velocidad de prototipado como la robustez del producto final. A continuación, exploraremos el primer flujo de trabajo: el enfoque eminentemente visual de Remix.

Análisis Comparativo de Flujos de Trabajo en Desarrollo de Smart Contracts: Remix IDE vs. Foundry Scripting

1.0 Flujo de Trabajo 1: El Enfoque Visual e Interactivo con Remix Desktop

La primera arquitectura se basa en la conexión de Remix Desktop, un Entorno de Desarrollo Integrado (IDE) visual, a un nodo blockchain Anvil que se ejecuta de forma aislada dentro de un contenedor Docker. En este modelo, Docker se encarga de proveer la infraestructura blockchain fundamental, mientras que Remix actúa como el "centro de mando" gráfico desde el cual se originan todas las interacciones con el contrato.

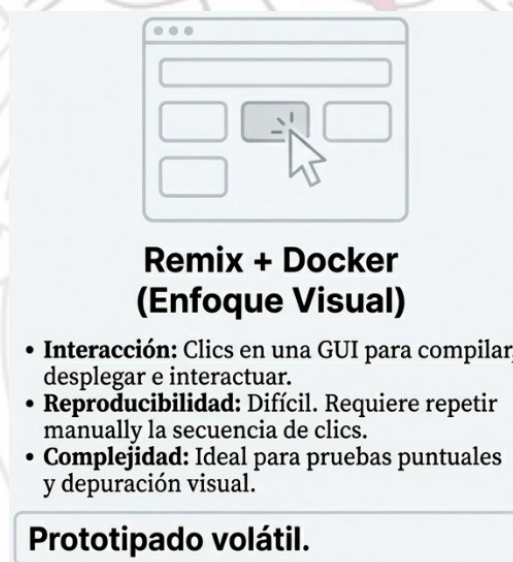


Figura 1: Clics

El funcionamiento de este flujo de trabajo es directo y se centra en la interacción manual a través de la interfaz de usuario:

- **Levantamiento del Nodo:** Se inicia el contenedor Docker que ejecuta el nodo Anvil, exponiendo su puerto RPC (generalmente el 8545) para permitir conexiones externas.
- **Conexión del IDE:** Dentro de Remix Desktop, el desarrollador se conecta al nodo local seleccionando la opción "External Http Provider" e introduciendo la dirección del nodo (<http://127.0.0.1:8545>).
- **Interacción Gráfica:** Una vez conectado, todo el ciclo de vida del contrato — compilación, despliegue e interacción con sus funciones— se gestiona haciendo clic en los botones correspondientes dentro de la interfaz gráfica de Remix.

Ventajas del Enfoque Visual

Este método presenta beneficios claros, especialmente en ciertas fases del desarrollo.

- **Feedback visual inmediato:** Remix genera automáticamente botones para cada una de las funciones del contrato, permitiendo al desarrollador invocar transacciones y ver los resultados al instante sin escribir una sola línea de script.
- **Curva de aprendizaje baja:** La simplicidad de "apuntar y hacer clic" lo convierte en una herramienta ideal para principiantes que están aprendiendo Solidity, así como para veteranos que necesitan realizar pruebas rápidas de una función aislada.
- **Debugger Visual:** Si una transacción falla, Remix ofrece un depurador visual que permite analizar la ejecución paso a paso, inspeccionar el estado de las variables y identificar la causa exacta del error de manera intuitiva.

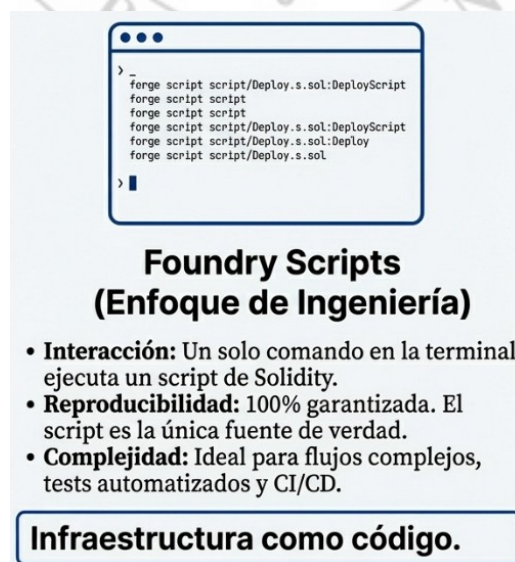
Desventajas Intrínsecas

A pesar de su accesibilidad, el enfoque de Remix presenta limitaciones significativas para el desarrollo profesional.

- **Manual y repetitivo:** Si el nodo Docker se reinicia, todo el estado de la blockchain se pierde. El desarrollador debe repetir manualmente cada paso: compilar, desplegar el contrato e invocar cada transacción en el orden correcto para restaurar el estado de prueba.
- **Difícil de reproducir:** Es sumamente complicado compartir un estado de prueba exacto con otros miembros del equipo. La descripción de los pasos se reduce a una lista de instrucciones manuales ("haz clic en Deploy, luego introduce este valor en la función deposit..."), un proceso propenso a errores y poco escalable.

Este enfoque manual y volátil contrasta directamente con la metodología programática y determinista que ofrece el ecosistema de Foundry, la cual analizaremos a continuación.

2.0 Flujo de Trabajo 2: El Enfoque de Ingeniería "Code-First" con Foundry



```

> _
forge script script/Deploy.s.sol:DeployScript
forge script script
forge script script
forge script script/Deploy.s.sol:DeployScript
forge script script/Deploy.s.sol:Deploy
forge script script/Deploy.s.sol
> █
  
```

Foundry Scripts
(Enfoque de Ingeniería)

- **Interacción:** Un solo comando en la terminal ejecuta un script de Solidity.
- **Reproducibilidad:** 100% garantizada. El script es la única fuente de verdad.
- **Complejidad:** Ideal para flujos complejos, tests automatizados y CI/CD.

Infraestructura como código.

Figura 2: Código

El ecosistema de Foundry representa un cambio de paradigma hacia un enfoque integral "code-first", donde cada aspecto del ciclo de vida del smart contract —desde la compilación y el despliegue hasta la ejecución de funciones complejas— se define y automatiza a través de código. Este método se ha consolidado como el estándar profesional para el desarrollo robusto, reproducible y escalable en el ecosistema de Ethereum.

El flujo de trabajo con Foundry es secuencial y se basa en un conjunto de herramientas de línea de comandos (CLI) que operan sobre una estructura de proyecto bien definida.

1. **Inicialización del Proyecto:** Se utiliza el comando `forge init`. Este comando no solo crea la estructura de carpetas (`src/`, `script/`, `test/`), sino que también inicializa un repositorio Git e instala la librería estándar `forge-std` como un submódulo. Se genera un archivo de configuración central, `foundry.toml`, que gestiona las dependencias, las reglas del compilador y los *remappings* (alias de importación) cruciales para resolver dependencias como `forge-std`.
2. **Desarrollo del Contrato:** El desarrollador escribe la lógica del negocio en archivos Solidity, como `PiggyBank.sol` o `PagoCondicional.sol`, que se ubican en el directorio `src/`.
3. **Creación de Scripts de Interacción:** En lugar de clics, se codifican las interacciones. Foundry ofrece dos modalidades principales:
 - **Scripts Monolíticos (`forge script`):** Se escribe un script en Solidity (ej. `script/Interact.s.sol` para `PiggyBank.sol`). Este script define una secuencia programática completa de acciones, utilizando las claves privadas de Anvil para simular múltiples actores (despliegue, depósito, transferencia). Es ideal para configuraciones de estado complejas y despliegues completos.
 - **Comandos Granulares (`forge create` y `cast`):** Para interacciones más atómicas, se utilizan comandos de CLI. Por ejemplo, con `PagoCondicional.sol`, se puede usar `forge create` para desplegar el contrato con argumentos de constructor específicos y luego `cast send` para invocar sus funciones (`reclamarPago`) de forma individual desde la terminal. Esto demuestra la versatilidad de Foundry para interacciones tanto programáticas como manuales por CLI.
4. **Ejecución y Automatización:** Con un único comando (`forge script ...` o una secuencia de `cast send ...`), Foundry compila los contratos, los despliega en el nodo Anvil y ejecuta toda la secuencia de transacciones. El proceso es automatizado y se completa en milisegundos.

Ventajas del Enfoque Foundry

Este modelo de ingeniería de software aporta beneficios cruciales para proyectos serios.

- **Reproducibilidad:** La reproducibilidad basada en scripts de Foundry es la piedra angular de cualquier pipeline de CI/CD profesional. Elimina los errores de tipo "funciona en mi máquina" y permite despliegues fiables y automatizados en `testnets` y `mainnet`. Un script es una garantía de un estado idéntico en cada ejecución.
- **Automatización:** Es la solución idónea para flujos de trabajo complejos (DeFi, gobernanza, etc.) que requieren desplegar múltiples contratos, configurar permisos y fondear pools. Replicar estas secuencias manualmente en Remix sería agotador y propenso a errores.
- **Testing Integrado y Rápido:** Foundry incluye un framework de pruebas nativo (`forge test`) que permite escribir pruebas unitarias en Solidity y ejecutarlas a velocidades extremadamente altas, facilitando la adopción de metodologías como el Desarrollo Guiado por Pruebas (TDD).

Prerrequisitos del Enfoque

El poder de Foundry viene acompañado de una curva de aprendizaje inicial.

- **Menos visual:** El feedback se produce principalmente a través de la consola. El desarrollador depende de los logs (`console.log`) y de herramientas CLI como `cast` para verificar el estado, en contraste con la inmediatez visual de Remix.
- **Requiere un mayor dominio de Solidity:** Este no es tanto una desventaja como un prerrequisito para la ingeniería de contratos a nivel de producción. Para aprovechar Foundry, es necesario escribir no solo la lógica del contrato, sino también los scripts de despliegue e interacción en Solidity, lo que implica un conocimiento más profundo del lenguaje y del ecosistema.

Ahora que hemos delineado ambos flujos de trabajo, podemos proceder a una comparación directa y estructurada de sus características clave.

3.0 Comparativa Directa: Remix vs. Foundry

Para tomar una decisión informada sobre qué herramienta utilizar, es fundamental evaluar ambos flujos de trabajo según criterios específicos que impactan directamente en la productividad y la calidad del desarrollo. Esta sección consolida las diferencias clave en un formato comparativo claro, permitiendo una evaluación objetiva de sus fortalezas y debilidades.

Resumen Comparativo de Flujos de Trabajo

Característica	Remix Desktop + Docker	Foundry Scripts (Puro)
Interacción	GUI (Gráfica): Clics y botones.	CLI (Consola): Comandos y código.

Velocidad de Prototipado	Muy rápida para probar 1 función.	Rápida para probar flujos completos.
Persistencia	Volátil (si cierras, repites clics).	Permanente (el script queda guardado).
Complejidad	Baja (Ideal para principiantes).	Media/Alta (Estándar profesional).
Caso de uso ideal	Aprender, depurar errores visualmente.	CI/CD, Desarrollar sistemas, Tests.

Análisis Detallado de las Características

En cuanto a la **Interacción**, la diferencia es fundamental. Remix ofrece una experiencia gráfica e intuitiva, ideal para quienes prefieren una aproximación visual. Foundry, por otro lado, se basa en una interfaz de línea de comandos (CLI), un enfoque preferido en flujos de trabajo de ingeniería de software por su capacidad para la automatización y el scripting.

Respecto a la **Velocidad de Prototipado**, existe una importante distinción. Remix es imbatible para probar una única función de forma aislada. Sin embargo, para un desarrollo sistémico, el ciclo de feedback casi instantáneo de `forge test` de Foundry habilita un flujo de Desarrollo Guiado por Pruebas (TDD) mucho más eficiente, acelerando el desarrollo del sistema completo a largo plazo.

La **Persistencia** del estado de prueba es una debilidad arquitectónica de Remix. Su naturaleza volátil obliga a reconstruir manualmente el entorno. Foundry resuelve esto de raíz: un script de despliegue puede ser versionado en Git, sometido a revisión de código (`code review`) y servir como documentación auditable del proceso de despliegue, una capacidad completamente ausente en el flujo de trabajo de Remix.

En términos de **Complejidad**, Remix es indiscutiblemente más accesible. Foundry, al requerir la escritura de scripts en Solidity y el manejo de la CLI, presenta una curva de aprendizaje más pronunciada, pero es una inversión necesaria para alcanzar el estándar de desarrollo profesional.

Finalmente, el **Caso de uso ideal** destila estas diferencias. Remix es la herramienta perfecta para el aprendizaje, la experimentación y la depuración visual. Foundry, con su enfoque en la reproducibilidad y la automatización, es la elección indiscutible para la construcción de sistemas de producción, la integración en pipelines de CI/CD y la ejecución de suites de pruebas exhaustivas.

La elección no debe ser mutuamente excluyente. De hecho, la estrategia más efectiva implica combinar lo mejor de ambos mundos.

4.0 El Enfoque Híbrido: La Estrategia Óptima para el Desarrollador Moderno

En la práctica, los equipos de alto rendimiento no eligen una herramienta sobre la otra de forma excluyente. El enfoque híbrido, que combina estratégicamente Foundry y Remix, se ha consolidado como el *de facto industry standard* para el desarrollo de smart contracts. Limitarse exclusivamente a Remix es un indicador de un enfoque junior; dominar el flujo de trabajo híbrido es una característica distintiva de la seniority.

Este flujo de trabajo óptimo se estructura de la siguiente manera:

1. **Desarrollo Central con Foundry:** Foundry se establece como la base del proyecto, la columna vertebral de la ingeniería de software. Se utiliza para la lógica principal, la gestión de dependencias, la ejecución de pruebas automatizadas y la creación de scripts de despliegue deterministas, garantizando que el proyecto sea robusto, reproducible y apto para la integración continua (CI/CD).
2. **Depuración y Pruebas Rápidas con Remix:** Cuando una prueba en Foundry falla o se necesita inspeccionar una transacción compleja, el desarrollador conecta Remix al mismo nodo local de Anvil. Desde Remix, puede invocar manualmente la función problemática, utilizar el depurador visual para analizar la ejecución paso a paso e identificar el error, o realizar una prueba manual rápida sin la sobrecarga de escribir un nuevo script.

Este enfoque combina la robustez y automatización de Foundry con la agilidad y el feedback visual de Remix. Este óptimo flujo de trabajo, sin embargo, se fundamenta en el poder del nodo subyacente. Para comprender por qué toda esta pila profesional se construye sobre Foundry, debemos analizar su motor central, Anvil, y entender su superioridad técnica sobre clientes de producción como Geth para fines de desarrollo.

5.0 Anexo Técnico: Por Qué Anvil es Superior a Geth para el Desarrollo Local

La elección de Anvil (el nodo incluido en Foundry) sobre un cliente de producción como Geth ([ethereum/client-go](https://github.com/ethereum/client-go)) es una decisión técnica fundamental que acelera drásticamente el ciclo de desarrollo. La siguiente analogía ilustra la diferencia: usar Geth para el desarrollo local es como construir un avión real en un garaje solo para probar si el diseño de los asientos es cómodo. Anvil, en cambio, es un simulador de vuelo avanzado que permite teletransportarse, cambiar la gravedad y detener el tiempo.

A continuación, se detallan las razones técnicas clave por las que Anvil es la opción recomendada.

5.1 Modelo de Minado: Determinismo vs. Realismo

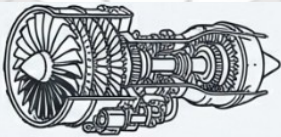
Geth está diseñado para imitar el comportamiento de la red real, lo que implica tiempos de bloque realistas (p. ej., 12 segundos). Esto introduce latencia en cada transacción, haciendo que la ejecución de pruebas sea un proceso lento. Anvil, por el contrario, utiliza un modelo de **minado instantáneo y bajo demanda**. El nodo crea un bloque en el milisegundo exacto en que recibe una transacción, permitiendo ejecutar miles de pruebas en segundos.

5.2 "God Mode": Cheatcodes y Control Total del Estado

Anvil expone **métodos JSON-RPC especializados** (conocidos como "cheatcodes") que permiten manipular el estado de la blockchain de maneras imposibles en un cliente de producción como Geth.

- **Suplantación de Cuentas (vm.prank):** Permite ejecutar una transacción *desde* cualquier dirección sin poseer su clave privada. Es invaluable para probar modificadores de acceso (onlyOwner) y lógica de permisos complejos.
- **Viaje en el Tiempo (vm.warp):** Permite avanzar el reloj de la blockchain instantáneamente. Si un contrato tiene un bloqueo de tiempo de un año, un simple comando puede simular el paso de ese año en un milisegundo.
- **Snapshots y Reversión de Estado (evm_snapshot / evm_revert):** Anvil permite guardar el estado completo de la blockchain en un punto, ejecutar una serie de transacciones complejas o destructivas y luego revertir instantáneamente al estado guardado. Esto es esencial para aislar pruebas y garantizar un entorno limpio para cada ejecución.

5.3 Configuración Cero vs. Bloque Génesis



Geth (Avión Real)

- **Minado:** Realista, con tiempos de bloque (ej. 12s) o configuración PoA compleja.
- **Control:** Reglas estrictas del protocolo. No se puede manipular el estado ni el tiempo.
- **Configuración:** Compleja, requiere 'genesis.json', creación de claves y banderas de arranque (using JetBrains Mono font).

Diseñado para operar en producción con máxima seguridad.

Figura 3: Avión Real

Levantar una red privada con Geth es un proceso laborioso que requiere crear claves, redactar un archivo `genesis.json`, inicializar la base de datos y arrancar el nodo. Anvil elimina toda esta fricción: un solo comando (`anvil`) inicia un nodo funcional con 10 cuentas pre-fondeadas con 10,000 ETH cada una, un límite de gas generoso y los puertos RPC listos para usar.

5.4 Forking de Mainnet: La Capacidad Definitiva para Pruebas de Integración



Figura 4: Simulador de Vuelo

Anvil permite crear una copia local de la blockchain de Ethereum Mainnet en un punto determinado. Esta funcionalidad descarga el estado actual de la red, incluyendo el código de todos los contratos desplegados (como Uniswap o USDC) y los saldos de las cuentas. Esto permite a los desarrolladores probar interacciones con protocolos reales de Mainnet en un entorno local, seguro y sin coste, una tarea que requeriría un esfuerzo monumental con Geth.

Esta superioridad técnica de Anvil como entorno de simulación refuerza su posición como la base ideal para los flujos de trabajo de desarrollo modernos.

Conclusión y Recomendaciones Finales

El análisis de los flujos de trabajo con Remix y Foundry revela que la elección entre ellos no es una cuestión de "mejor" o "peor", sino de adecuación a la tarea, la fase del proyecto y la experiencia del desarrollador. Cada ecosistema ofrece un conjunto único de ventajas que, cuando se aplican como parte de una estrategia cohesiva, mejoran drásticamente la productividad y la calidad del código.

A continuación, se presentan los siguientes patrones arquitectónicos y mejores prácticas:

- **Use Remix Desktop cuando:**
 - Esté en las primeras etapas de aprendizaje de Solidity y necesite un entorno para experimentar.
 - Necesite depurar visualmente una transacción compleja para entender su flujo de ejecución paso a paso.
 - Quiera prototipar rápidamente la lógica de una única función de forma aislada.
- **Use Foundry cuando:**
 - Esté construyendo un sistema de producción que requiera robustez, fiabilidad y mantenimiento a largo plazo.
 - Necesite una suite de pruebas automatizadas que se ejecuten de manera rápida y consistente como parte de un pipeline de CI/CD.
 - Requiera despliegues reproducibles y auditables.
 - Esté trabajando en un proyecto colaborativo donde la coherencia del entorno es crucial.
- **Adopte el Enfoque Híbrido como estándar:**
 - La estrategia más eficaz es utilizar Foundry como la base para toda la ingeniería de software del proyecto —gestión de código, pruebas y scripting—. Al mismo tiempo, se debe mantener Remix como una herramienta complementaria y especializada para la depuración puntual y la inspección visual del estado.

En última instancia, dominar ambos flujos de trabajo y desarrollar la intuición para saber cuándo aplicar cada uno no solo optimiza el proceso de desarrollo, sino que también es una marca distintiva de un desarrollador de smart contracts senior, versátil y altamente eficaz.

Bibliografía

- **Foundry — documentación oficial (Forge / Anvil / Cast / Chisel)** — guía de instalación, testing, scripting, cheatcodes y forking (muy relevante para el enfoque *code-first* y pruebas locales con forking). getfoundry.sh⁺²
Por qué: Foundry es el estándar de facto para workflows rápidos basados en Rust/Forge; imprescindible si vas a profundizar en scripting, tests y forking.
- **Remix IDE — documentación y tutoriales (Remix Desktop / LearnEth)** — tutoriales interactivos, plugins, pruebas rápidas en navegador y entorno Desktop. remix-ide.readthedocs.io⁺¹
Por qué: Excelente para prototipado visual, debugging paso a paso y aprendizaje rápido de Solidity sin configurar toolchains.
- **Anvil (parte de Foundry) — reference / comandos** — descripción de Anvil como nodo local rápido, opciones de fork y RPC. getfoundry.sh⁺¹
- **Geth / Clientes Ethereum — comparativa devchains vs nodos locales** — diferencias de objetivo y capacidades (Geth como cliente realista; no tiene “cheatcodes” integrados). Útil para entender por qué Anvil es preferido por muchos desarrolladores para testing. [Ethereum Stack Exchange](https://ethereum.stackexchange.com)⁺¹